

# Array Data-Flow Analysis and its Use in Array Privatization

Dror E. Maydan, Saman P. Amarasinghe and Monica S. Lam  
Computer Systems Laboratory  
Stanford University, CA 94305

## Abstract

Data-flow analysis of scalar variables and data dependence analysis on array elements are two important program analyses used in optimizing and parallelizing compilers. Traditional data-flow analysis models accesses of array elements simply as accesses to the entire array, and is inadequate for parallelizing loops in array-based programs. On the other hand, data dependence analysis differentiates between different array elements but is flow-insensitive.

This paper studies the combination of these two analyses—data-flow analysis of accesses to individual array elements. The problem of finding precise array data-flow information in the domain of loop nests where the loop bounds and array indices are affine functions of loop indices was first formulated by Feautrier. Feautrier's algorithm, based on parametric integer programming techniques, is general but inefficient. This paper presents an efficient algorithm that can find the same precise information for many of the programs found in practice. In this paper, we argue that data-flow analysis of individual array elements is necessary for effective automatic parallelization. In particular, we demonstrate the use of array data-flow analysis in an important optimization known as array privatization.

By demonstrating that array data-flow analysis can be computed efficiently and by showing the importance of

the optimizations enabled by the analysis, this paper suggests that array data-flow analysis may become just as important in future optimizing and parallelizing compilers as data-flow and data dependence analysis are in today's compilers.

## 1 Introduction

Program analysis plays a crucial role in a compiler as it defines the kind of information that is made available to the optimizers. For a program analysis to be effective, it must capture sufficient information to enable sophisticated optimizations and be amenable to efficient implementation. Previous research on scalar compiler optimizations are based primarily on data-flow analysis while research on parallelizing compilers are based primarily on data dependence analysis. This paper explores a less understood domain of program analysis, the data-flow analysis of individual array elements. We show that such analysis is viable and useful.

In traditional data-flow analysis, accesses of array elements are simply modeled as accesses to the entire array. Such analysis is inadequate, for example, to detect parallelism within loops that access arrays. Data dependence is a powerful analysis whose results enable parallelization of loops as well as various forms of loop transformations. The domain of data dependence analysis is that the array index functions and the loop bounds must be integer linear functions of loop indices and possibly other symbolic variables. The traditional formulation of the data dependence problem can best be described as memory disambiguation. Specifically, data dependence asks if *any* of the dynamic instances of two array accesses in a loop nest may refer to the same *location*. This problem has been shown to be equivalent to integer programming. Fortunately, recent results show that the data dependence problems that exist in most programs

---

This research was supported in part by DARPA contracts N00039-91-C-0138 and DABT63-91-K-0003, and by fellowships from AT&T Bell Laboratories and Intel Corporation.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

ACM-20th PoPL-1/93-S.C., USA

© 1993 ACM 0-89791-561-5/93/0001/0002...\$1.50

can be solved exactly and efficiently[16].

Memory disambiguation does not capture some important information in programs. Consider the example below:

```
for i = 1 to 10 do
  a[i] = 0;      (S1)
  a[i] = 1;      (S2)
  ... = a[10];   (S3)
```

a data dependence analyzer will say that there is a dependence from statement *S1* to *S2*, a dependence from *S1* to *S3*, and a dependence from *S2* to *S3*. This information is sufficient to determine that it is not legal to execute all the iterations of the loop in parallel. However, it is imprecise in two ways:

*Coverage.* A data dependence analyzer does not know if a write operation covers, or kills, an earlier write operation. In this example, precise data-flow analysis of the individual array elements will find that the  $i^{\text{th}}$  definition in statement *S2* kills the  $i^{\text{th}}$  definition in statement *S1*. This information, for example, can be used to eliminate statement *S1*.

*Identity of dependence.* The data dependence analyzer only knows that a dependence exists; it does not know which pairs of instances are dependent. Precise data-flow analysis can determine that only the tenth instance of statement *S3* depends on the tenth instance of *S2*. This information can be used to parallelize the first nine iterations of the loop.

Recent research in parallelization has uncovered the need for new optimizations that require program information beyond that provided by either scalar data-flow or data dependence analysis. For example, it is well known that *array privatization* is crucial for automatic parallelization to succeed[7, 17, 22]. A common programming practice is to use the same array to store working data across different iterations of a loop. Such reuse renders the loop unparallelizable. Evaluation of today's compiler technology indicates that many more loops can be parallelized by privatizing these work arrays, that is, assigning a separate copy of the array to each processor. To determine if each iteration can have its own copy of the array, we need *data-flow analysis* on individual array elements. We need to know if any of the iterations requires the definitions in the preceding iterations of the same loop; we also need to know how to initialize the private copy of the array and how to determine the final values of the original array.

The need for data-flow analysis techniques is even more obvious in the compilation for message-passing machines. The high communication cost on these machines makes it necessary to minimize communication.

Critical optimizations include reducing total communication volume by eliminating redundant data movements and reducing start-up overhead per message by blocking the communication. Such optimizations again need data-flow information for individual array elements.

There have been two major approaches to finding data-flow information for array elements. The first builds on scalar data-flow analysis, and the second on data dependence.

The first approach is based on the scalar data-flow framework[12, 13, 14, 21]. Instead of representing an array with a single bit, the set of data touched within a region/interval in the flow graph is approximated by an area descriptor. These approaches handle arbitrary control flow, but their accuracy is defined by the precision of the summary information.

The operators in the data-flow equations are set operations (e.g. intersection, union and difference) on area descriptors. There are two difficulties with this approach. First, these descriptors may not be closed under these operations; the results must therefore be (conservative) approximates of the exact answer. Second, these area descriptors are typically expressions of symbolic variables. Solving this problem exactly requires the same parametric integer programming techniques used in the second approach. In fact, solving these equations in a bottom-up manner, as in interval analysis, would require the algorithm to handle outer loop indices initially as symbolic variables. This would unnecessarily make the algorithm more expensive.

The scalar data-flow framework generally associates the same data-flow information with all instances of an access in the program. To capture the "identity" of dependences requires distinguishing between different instances; such distinction is not within the vocabulary of the static data-flow framework. For example, this approach will summarize the data-flow information for the above code as writing  $A[1:10]$  and reading  $A[10]$ ; it will not detect that the first nine iterations of the loop do not read any of the values written in the loop.

The second approach, pioneered by Feautrier, uses the same framework as data dependence analysis. Feautrier developed an algorithm that finds perfect data-flow information for arrays in the domain of loop nests where the loop bounds and array indices are affine functions of the loop indices[9, 10, 11]. While Feautrier's solution does not handle general control flow directly, his techniques can be used to solve the fundamental problems that must be addressed by any exact array data-flow analysis. Other works tend to be more efficient, but they are all either less accurate or only applicable in more limited domains. Brandes' approach does not apply if dependence distances are coupled or if the loop is non-rectangular[5]. Ribas only discusses constant-distance dependences in perfectly nested loops[20]. Pugh and Wonnacott's algorithm is inaccurate when their Omega

test generates disjunctions of constraints[19]. It cannot handle cases when a read statement is covered by write instances from different loop nesting levels. In addition, it may have difficulties when the data touched by an access are not contiguous (i.e. a reference such as  $a[2i]$ ).

Array privatization and optimizations for message passing machines require the precision of Feautrier's algorithm. Unfortunately, our experience with our implementation of his algorithm suggests that it is too inefficient to use in practice. By exploiting the simplicity in real code, we have developed an efficient algorithm that gives the same precise information as Feautrier's in most of the cases found in practice. We believe that the efficiency of this algorithm is important in making array data-flow analysis feasible to use in program optimizations.

This paper also develops an array privatization algorithm that uses the data-flow information. Using the coverage information provided by data-flow analysis, we have developed the concept of *data-flow dependence vectors*, the data-flow analog of dependence vectors in the parallelization literature. We show how we use these vectors to determine when it is necessary and useful to privatize an array. Using the identity of the dependences, we can derive the *upwards-exposed* uses and *downwards-exposed* definitions information. This information is necessary in array privatization to initialize the private array and to finalize the original array.

## 2 Last Write Trees

The precise data-flow dependence relationships are captured in a representation we call a Last Write Tree (LWT). We first motivate this representation with a simple example:

```

for i = 11 to 20 do
  for j = 11 to 20 do
    a[j] = ...
    ... = a[j-1]
  
```

It is easy to see that the data read in the first iteration of the innermost loops ( $j=11$ ) are not defined within the loop nest. For all other iterations, the data read are written in the very preceding iteration. This information can be organized in a binary tree as shown in Figure 1. We call this a Last Write Tree (LWT); the tree represents a function that maps an instance of the read operation to the last instance of a write to the same location. The read and write instances are denoted by the values of their loop indices,  $\vec{i}_r$  and  $\vec{i}_w$ , respectively. The domain of the function is the set of  $\vec{i}_r$  that satisfy the constraints imposed by the loop bounds. Each internal node in the tree contains a further constraint on the value of the read instance  $\vec{i}_r$ . It partitions the domain of read instances into those

satisfying the constraint, represented by its right descendants, and those not satisfying the constraint, represented by its left descendants. In other words, the constraints of a node's ancestors make up the node's *context*. The *iteration set* of a node is defined as the set of  $\vec{i}_r$  that satisfy the node's context. Each leaf node has a solution which expresses the last write instance  $\vec{i}_w$  in terms of  $\vec{i}_r$ , for all those  $\vec{i}_r$  in its iteration set. If there is no preceding dependent write for a particular iteration set, the solution of the corresponding leaf is denoted by  $\perp$ .

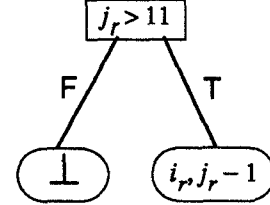


Figure 1: Example of an LWT

The LWTs provide information on both coverage and identities of dependences, the two deficiencies in standard data dependence analysis. First, the LWT is a function that specifies only the last write instance for a particular read instance, instead of specifying all previous writes to the same location. Second, the context of the LWT defines the identities of the dependent pairs. The LWT function is defined formally below.

**Definition 2.1** Let  $\vec{i}$  and  $\vec{i}'$  be iterations of two statements that are nested in  $n$  common loops and an arbitrary number of non-common loops. Vector  $\vec{i}$  is lexicographically less than vector  $\vec{i}'$ , written  $\vec{i} < \vec{i}'$ , iff

$$\exists k \leq n, i_1 = i'_1, \dots, i_{k-1} = i'_{k-1}, i_k < i'_k$$

Similarly,  $\vec{i} \leq \vec{i}'$  if  $\vec{i} < \vec{i}'$  or  $i_1 = i'_1, \dots, i_n = i'_n$ .

**Definition 2.2** Let  $\vec{i}_w, \vec{i}_w'$  be iterations of a write statement in an  $m$ -deep loop nest,  $\vec{i}_r$  be an iteration of a read statement in an  $n$ -deep loop nest, and  $\vec{s}$  be a  $p$ -element vector of symbolic constants used in the loop. Let

$$\begin{aligned} \vec{u}_w &= (i_{w1} \dots i_{wm}, s_1 \dots s_p), \\ \vec{u}_w' &= (i'_{w1} \dots i'_{wm}, s_1 \dots s_p), \\ \vec{u}_r &= (i_{r1} \dots i_{rn}, s_1 \dots s_p). \end{aligned}$$

Let  $F_w, F_r$  be affine read and write access functions in the loop, and  $B_w, B_r$  be affine read and write loop bound constraints.

If  $F_w$  appears after  $F_r$  lexically in the program, then  $\forall \vec{u}_r$  such that  $B_r \vec{u}_r \geq \vec{0}$ ,

$LWT_{F_w, F_r, B_w, B_r}(\vec{u}_r) = \vec{i}_w$  if and only if

$$B_w \vec{u}_w \geq \vec{0}, F_w \vec{u}_w = F_r \vec{u}_r, \vec{i}_w < \vec{i}_r, \text{ and}$$

$\nexists \vec{i}_w' (\neq \vec{i}_w)$  such that

$$B_w \vec{u}_w' \geq \vec{0}, F_w \vec{u}_w' = F_r \vec{u}_r, \vec{i}_w < \vec{i}_w' < \vec{i}_r$$

If instead, the write statement appears lexically above the read, we must replace the condition  $\vec{i}_w < \vec{i}_r$  in the definition with the condition  $\vec{i}_w \preceq \vec{i}_r$ .

### 3 Calculating LWTs

Feautrier has developed a set of algorithms, based on parametric integer programming, that can be used to calculate LWTs in the domain of loop nests that contain no IF statements and no procedure calls[9, 10, 11]. Feautrier's algorithms require solving an exponential number of memory disambiguation problems. Feautrier found that his algorithm takes 1.7 seconds to solve a simple input with one write statement and four read statements on a low end SPARC[11]. We have also implemented Feautrier's algorithms; we found them complex and obtained similar timing figures. We believe that the algorithm is much too inefficient to be used in practice.

The algorithm described below is designed to exploit the simplicity in real code. It can find exact data-flow information for the vast majority of programs very efficiently. On Feautrier's example that took 1.7 seconds, our algorithm takes about 0.10 seconds on a similar machine, a DECstation 3100, to both calculate the LWT and to compute the data-flow dependence vectors described in Section 4. The efficiency of our algorithm makes it feasible to use array data-flow in program optimizations.

In the following, we first concentrate on finding the array data-flow information between one pair of read and write operations in a loop nest with affine loop bounds. To explain the algorithm, we describe our observations on two degenerate cases: when a write access touches the same locations in all instances and when a write access touches all different locations in different instances. We then show how we can use these ideas as building blocks to develop an algorithm applicable to many of the cases found in practice. We will back up the claim of its generality by some empirical evidence. Finally, we describe how to handle multiple write operations.

#### 3.1 Loop Independent References

We say that a reference is *loop-independent* if it refers to the same location in every iteration. In the following example:

```
do  $i_1 = 11$  to 15
  do  $i_2 = 11$  to 15
    ... =  $a$ 
     $a = \dots$ 
  end do
end do
```

every read and write iteration refers to the same location. The last *dependent* write before a read instance is simply

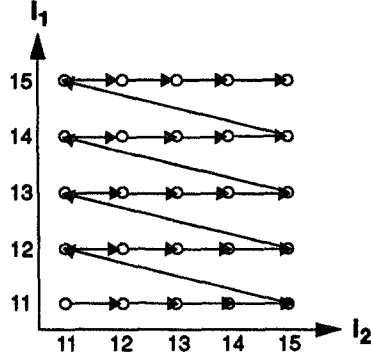
the last write before the read instance. Except for the first iteration which reads an uninitialized value, every iteration reads the data written in the immediately preceding iteration. Figure 2(a) illustrates the iteration space ordering for the example. From the figure, we observe that if the second loop index of the read iteration,  $i_{r_2}$ , is not at its minimum value, 11, then one iteration before the read is  $(i_{r_1}, i_{r_2} - 1)$ . Otherwise, as long as the read iteration is not the first iteration of the loop ( $i_{r_1} > 11$ ), one iteration before the read is  $(i_{r_1} - 1, 15)$ . This information is represented by the LWT in Figure 2(b).

We now describe the general procedure for finding the LWT of a loop-independent read and write that accesses the same location and are perfectly nested in  $n$  common loops. We first assume that every loop has at least one iteration. Let  $\vec{i}_w$  be the last dependent write for read iteration  $\vec{i}_r$ . If the write statement appears lexically before the read, then  $\vec{i}_w = \vec{i}_r$ . If the read statement appears lexically before the write statement, we create a tree similar to the one in the example using a simple, recursive procedure. We start our recursion with the innermost loop. At any level  $k$  in the recursion, we create a subtree with the root "if  $i_{r_k} > LB_k$ ", where  $LB_k$  is the expression for the lower bound of  $i_{r_k}$ , and where  $i_{r_k}$  refers to the  $k^{\text{th}}$  component of  $\vec{i}_r$ . When  $k = 1$ , the false child is simply  $\perp$ ; otherwise, the false child is the result of applying the recursive step to level  $k - 1$ . The true child is the solution node  $i_{w_{1..k-1}} = i_{r_{1..k-1}}, i_{w_k} = i_{r_k} - 1, i_{w_{k+1..n}} = UB_{k+1..n}$ , where each  $UB_l$  is the upper bound for  $i_{w_l}$ . If the loop is trapezoidal, that is,  $UB_l$  is an expression of outer loop indices, we substitute in the values of the other components of  $\vec{i}_w$  to find its value.

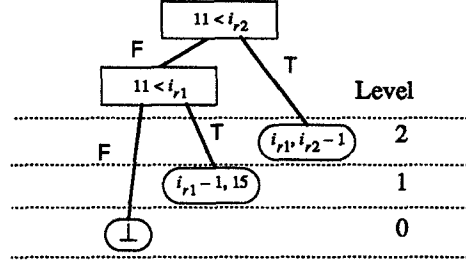
If the loops are not perfectly nested, a write access may have additional inner loop indices; however, these indices do not affect the relative ordering between the write and read instances. Suppose a write statement is nested in  $m$  loops and shares only the outer  $n$  loops with the read statement. We first apply the procedure above on the commonly nested loops, then for every non- $\perp$  solution, we set  $i_{w_{n+1}}, \dots, i_{w_m}$  to be their upper bounds.

We now consider the possibility of having *degenerate* loop nests, nests where some of the loops may have zero iterations. A loop is *fully degenerate* if it will either always have iterations or never have iterations depending on the values of the symbolic constants. A loop is *partially degenerate* if it is degenerate only for some values of the outer loop indices. In the example

```
do  $i = 1$  to 10
  do  $j = 1$  to  $n$ 
     $a = \dots$ 
  end do
end do
```



(a)



(b)

Figure 2: LWT for loop-independent references

the second loop is fully degenerate: it will always have at least one iteration if  $n \geq 1$ , and none otherwise. In contrast,

```

do  $i_1 = n$  to 15
  do  $i_2 = 12$  to  $i_1$ 
    ... =  $a$ 
     $a = \dots$ 
  end do
end do

```

the second loop here is partially degenerate, because it has no iterations only when  $i_1 < 12$ .

Ancourt has developed an algorithm for loops with affine loop bounds based on Fourier-Motzkin elimination that can be used to detect possibly degenerate loops [2]. For full degeneracies, we can use the algorithm to derive the conditions under which the loop will execute, i.e.  $n \geq 1$  in the above example. We merely need to guard our LWT with these conditions; if these conditions are not satisfied, no writes are executed and the solution is  $\perp$ .

Our LWT algorithm relies on being able to find the first iteration of each loop, and thus cannot tolerate partial degeneracies. Ancourt's algorithm can also rewrite the loop bounds to eliminate partial degeneracies; e.g. it will replace the lower bound of  $i_1$  in the partial degeneracy example with the value  $\max(n, 12)$ . This procedure solves the partial degeneracy problem if the read and write statements are nested in the same loops. If the read and write statements have different enclosing loops, however, eliminating the partial degeneracy in one of these different loops may change the actual bounds of a common loop for one of the statements. Our algorithm cannot handle this situation, and Feautrier's algorithm should be used. Fortunately, as shown in Section 3.3.1, this type of degeneracy is rare.

### 3.2 Writes That Do Not Self-Interfere

We now consider the opposite scenario where a write operation accesses a different location in each iteration. We say that such a write does not “self-interfere”. A write that never self-interferes writes at most one value into any location. For such writes, computing the last dependent write before a read is equivalent to computing all the dependent writes before the read; we can easily adapt our affine memory disambiguation system to generate the desired LWT.

To simplify the presentation in this section, we assume that no symbolic constants are used in the access functions, and that the access function  $F_w$  is invertible. Thus, for there to be a dependence, the read and write must refer to the same location. i.e.,  $F_w \vec{i}_w = F_r \vec{i}_r$ , or  $\vec{i}_w = F_w^{-1} F_r \vec{i}_r$ . The domain of the LWT is the set of read iterations,  $\vec{i}_r$ , that satisfy the constraints of the loop bounds,  $B_r \vec{i}_r \geq 0$ . The dependent writes must be within the write bounds  $B_w \vec{i}_w \geq 0$ , and that these writes must execute before the read. The LWT template in Figure 3 ensures that these constraints are met for the case where the read statement appears lexically before the write statement.

Assume that the write statement is nested in  $m$  loops,  $n$  of which are common to the read. After expanding out the lexicographical ordering constraint and replacing in the conditional each occurrence of  $\vec{i}_w$  with its value,  $\vec{i}_w = F_w^{-1} F_r \vec{i}_r$ , we get the LWT in Figure 4. Note that our lexicographic ordering constraint is expanded into two internal nodes for each of the  $n$  common loop indices, but our solution  $\vec{i}_w = F_w^{-1} F_r \vec{i}_r$  is a solution for all the write loop indices  $i_{w_1}, \dots, i_{w_m}$ . Note also that if the write statement appears lexically before the read, the true child of the lowest internal node,  $(F_w^{-1} F_r \vec{i}_r)_n \geq i_{r_n}$ , will be  $F_w^{-1} F_r \vec{i}_r$  rather than  $\perp$ .

Some nodes of the tree may be inconsistent. If, for

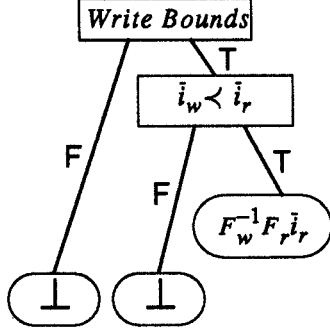


Figure 3: Template LWT for non-self-interfering write

example, there is no dependence such that  $i_{w_n} \geq i_{r_n}$ , the last condition is always false. We can prune away this node, replacing the subtree rooted at  $i_{w_n} \geq i_{r_n}$  with its false child. We can find the inconsistent nodes by testing each of the nodes for consistency. Checking all the nodes for consistency is relatively inexpensive. This is similar to finding if there is a data dependence for each dependence level, which is only a subset of the standard data dependence analysis.

In general,  $F_w$  and  $F_r$  are affine functions of the loop indices and a symbolic constant vector. Thus,

$$F_w \vec{u}_w = F_w^1 \vec{i}_w + F_w^2 \vec{s} + \vec{c}_w, \text{ and}$$

$$F_r \vec{u}_r = F_r^1 \vec{i}_r + F_r^2 \vec{s} + \vec{c}_r,$$

for some integer matrices  $F_w^1, F_w^2, F_r^1, F_r^2$ . Since  $F_w \vec{u}_w = F_r \vec{u}_r$ , thus  $F_w^1 \vec{i}_w = G(\vec{u}_r)$ , where

$$G(\vec{u}_r) = F_r^1 \vec{i}_r + (F_r^2 - F_w^2) \vec{s} + \vec{c}_r - \vec{c}_w.$$

We can use the Smith normal form to express  $\vec{i}_w$  in terms of  $\vec{u}_r$ . In a process similar to the extended GCD Test [4], which is similar to an integral version of LU decomposition, we can find matrices  $P, Q$  and  $S$  such that  $PF_w^1 Q = S$ ,  $P$  and  $Q$  are unimodular matrices and  $S$  is of the form  $\begin{bmatrix} D & 0 \\ 0 & 0 \end{bmatrix}$  where  $D$  is a positive diagonal matrix. The initial system has an integer solution if and only if there exists an integer vector  $\vec{t}$  such that  $S\vec{t} = PG\vec{u}_r$ . Since  $D$  is a diagonal matrix, the constraint  $S\vec{t} = PG\vec{u}_r$  is equivalent to the set of constraints  $D_{x,x} t_x = (PG\vec{u}_r)_x$ . An integral  $t_x$  satisfying this constraint exists if and only if  $(PG\vec{u}_r)_x \% D_{x,x} = 0$  where  $\%$  is the integer modulo operation.

If there exists an integral solution, then the solution is given by  $\vec{i}_w = Q\vec{t}$ . If there are no zero diagonal elements in  $S$  ( $S = D$ ),  $\vec{t} = D^{-1}PG\vec{u}_r$ , where  $D^{-1}$  is a diagonal matrix such that  $(D^{-1})_{x,x} = 1/D_{x,x}$ . To create the LWT, we merely add to our tree the nodes  $(PG\vec{u}_r)_x \% D_{x,x} = 0$ . If these conditions are not met,

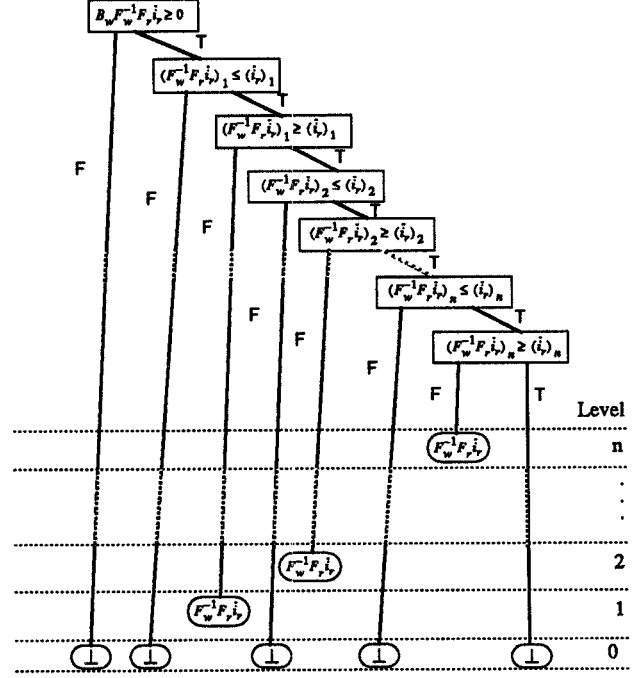


Figure 4: LWT for non-self-interfering write

there is no dependent write, otherwise we create a tree as shown in Figure 4, replacing  $\vec{i}_w = F_w^{-1} F_r \vec{i}_r$  with  $\vec{i}_w = QD^{-1}PG\vec{u}_r$ .

If  $S$  has a 0 in the diagonal, there are multiple values of  $\vec{i}_w$  for each  $\vec{u}_r$  that satisfy  $F_w^1 \vec{i}_w = G(\vec{u}_r)$ . However, for the write to qualify as not self-interfering, the loop bounds must impose sufficient additional constraints on the problem to reduce the solution to at most one single value of  $\vec{i}_w$  for each  $\vec{u}_r$ . For these rather uncommon cases, we can use a memory disambiguator to determine the unique  $\vec{t}$  [18].

Note that degenerate loops are not a problem for this case. If a particular solution for  $\vec{i}_w$  is invalid due to a degeneracy, one of the loop bound constraints will fail, and our solution will be  $\perp$ .

### 3.3 Computing LWT Efficiently

We have shown how to construct LWTs for pairs whose write statements do not self-interfere and for pairs with loop-independent references. Both of these cases, by themselves, are not very interesting. When write statements do not self-interfere, LWTs contain no more information than standard memory disambiguation. The advantage of data-flow dependence analysis is to eliminate dependences that are covered by other writes. Also, not many writes are loop-independent. We can, though, use

|        | AP  | CS | LG | LW | MT | NA  | OC | SD  | SM  | SR  | TF  | TI | WS  | TOTAL |
|--------|-----|----|----|----|----|-----|----|-----|-----|-----|-----|----|-----|-------|
| SI     | 0   | 2  | 0  | 0  | 0  | 13  | 39 | 0   | 0   | 0   | 0   | 12 | 0   | 66    |
| Non-SI | 424 | 84 | 36 | 36 | 74 | 316 | 75 | 142 | 124 | 430 | 233 | 10 | 517 | 2,501 |

Table 1: Number of self-interfering (SI) and non-self-interfering (non-SI) writes after removing unused indices

our algorithms for these two cases as building blocks for an algorithm that is both simple and widely applicable.

### 3.3.1 Domain of Algorithm

To specify the domain of our algorithm precisely, let us first introduce the notion of *unused loop indices*. We define a loop index to be unused if it does not appear in the array reference function of the write nor in the bounds of any used loop index. An unused index may be used by the read statement. A loop-independent write corresponds to the degenerate case when all the loop indices are unused by the write.

Our algorithm can generate the LWT exactly and efficiently if eliminating the unused loop indices reduces the write access into a non-interfering one, and if Ancourt’s algorithm to eliminate partial degeneracies does not introduce read bounds that are different from the write bounds in any of the removed loops. To assess the generality of our algorithm, we performed the following experiment. We inserted a pass in our SUIF (Stanford University Intermediate Format) compiler to examine all of the array writes in the PERFECT Club programs that have affine array index expressions and are nested in at least one loop. After eliminating the unused indices, we checked to see if each write statement self-interfered. This is equivalent to checking for an output memory disambiguation dependence from the write to itself with a non-zero direction vector component. Table 1 shows that after removing the unused indices, 2,501 out of 2,567 (over 97%) writes do not self-interfere.

Some of these writes may not have been self-interfering even before eliminating the unused variables and, for these cases, our analysis generates no more information than standard memory disambiguation. Even looking only at writes which do originally self-interfere, we are able to handle 566 out of 632 (about 90%) cases.

We also performed a simple experiment to determine the frequency of partial degeneracies. Partial degeneracies can only occur in trapezoidal loops. For each trapezoidal loop, we used a simple and efficient substitution step to determine if a trapezoidal loop can be partially degenerate[15]. Our simple algorithm was able to prove that over 98% of the writes in the PERFECT Club were not nested in any partially degenerate loops.

### 3.3.2 Our Algorithm

The outline of our algorithm is as follows:

1. Apply Ancourt’s algorithm to find the conditions for full degeneracy and to eliminate partial degeneracies. The conditions for full degeneracy are placed at the top of our LWT.
2. Remove the loops corresponding to those loop indices that are unused by the write statement. We treat the unused loop indices remaining in the read statements as symbolic constants.
3. Check if the write in the reduced system self-interferes. If the write self-interferes, the algorithm is not applicable. Otherwise:
4. Construct the LWT for the reduced system using the non-self-interfering algorithm.
5. Construct the LWT for the original system by *restoring* the unused loop indices. We will describe this step in detail below. If Ancourt’s algorithm has set any of the write bounds in the unused loops indices to be different from the corresponding read bounds, this step of our algorithm is not applicable, and we must use Feautrier’s algorithm to restore the unused indices.

Note that even when our algorithm does not apply because of self-interference, our LWT is only inaccurate in that it conservatively describes more dependent writes than just the last write before the read. Our algorithm is still useful in pruning some of the false data-flow dependences in cases where only some of the writes to the same locations occur in iterations whose loop indices are unused.

### 3.3.3 Restoring Unused Variables

To construct the LWT for the original system from the LWT for the reduced system, our algorithm restores the unused indices one at a time starting with the innermost index. We show, at each stage, that our algorithm generates the correct tree for the partially reduced system with all loops corresponding to the not yet restored variables removed.

Recall that the values of the non-common indices do not affect the ordering between the read and write. Therefore, for any non- $\perp$  solution node, we should set all non-common, unused write indices to their upper bounds, and

we should set all the non-common, used indices to their corresponding component in the solution,  $QD^{-1}PG\vec{u}_r$ . We can ignore the non-common loops for the remainder of our discussion.

The algorithm to restore the index corresponding to the  $k^{\text{th}}$  outer loop is as follows. Let  $T$  be the LWT before the restoration and  $T'$  be the new tree. Each internal node in an LWT partitions the domain of the LWT function. A given value of  $\vec{i}_r$  and symbolic variables falls in the iteration set of one and only one leaf. Thus, when creating  $T'$ , we must merely ensure that each leaf contains the correct solution for all the reads in its read iteration set. We consider non- $\perp$  and  $\perp$  leaves in  $T$  separately below.

**non- $\perp$  leaves in  $T$ :** Every read in the read iteration set of a non- $\perp$  leaf in the fully reduced system is preceded by one and only one write to the same location. By definition, the unused indices do not affect the location being written. Therefore, the location written in  $\vec{i}_w$  in the full system is the same as the location read in  $\vec{i}_r$  if and only if the solution for all the used indices in the full system is the same as the solution for all these indices in the reduced system. Thus to restore the unused index,  $i_{w_k}$ , to the leaf in  $T$ , we merely need to set the unused indices so that  $\vec{i}_w$  is the last write before the read.

Each non- $\perp$  leaf in  $T$  has a level,  $l$ , corresponding to the minimum index such that  $i_{w_l} < i_{r_1}$ . If no such index exists, then  $\vec{i}_w = \vec{i}_r$  in  $T$ , and we define  $l = n + 1$  where  $n$  is the number of common loops in the original, full system. Note that since we are restoring the index for the  $k^{\text{th}}$  outer loop, there are no leaves with level  $l = k$  in  $T$ . Thus the two possible cases are:

1.  $l < k$ . Since the algorithm restores the innermost unused indices first, and since index  $l$  appears in  $T$ ,  $i_{r_1}$  must correspond to a used index. Since the restoration process cannot change the value of a used index, we know that  $i_{w_l} < i_{r_1}$  in  $T'$ . Whatever value we give  $i_{w_k}$  in  $T'$ , the write will come before the read. We therefore set  $i_{w_k}$  to be as large as possible, the upper bound for loop index  $i_k$ ,  $UB_k(i_{w_1}, \dots, i_{w_{k-1}})$ . If the upper bound is a constant, we simply set  $i_{w_k}$  to its value. Otherwise, if the loop is trapezoidal, we can use simple substitution to find its value. Note that we must ensure that any previously restored variable retains its correct value throughout the restoration process. Since any previously restored variable,  $i_{w_m}$ , has  $m > k$ , we know that  $m > l$  and therefore  $i_{w_m}$  has previously been set to its upper bound, which is still its correct value.
2.  $l > k$ . Since in  $T$ ,  $\vec{i}_w \preceq \vec{i}_r$ , we must have  $i_{w_1} = i_{r_1}, \dots, i_{w_{k-1}} = i_{r_{k-1}}$  for all the existing indices in  $T$ . If we set  $i_{w_k}$  to be larger than  $i_{r_k}$ ,

then the write instance will be later than the read so we set  $i_{w_k} = i_{r_k}$ . This is always possible since the bounds on the read are equivalent to the bounds on the writes. Now consider once again each previously restored variable,  $i_{w_m}$ . Our restoration process has not changed the level of the leaf; it is still  $l$ . Since the value of  $i_{w_m}$  was determined solely by the relative values of  $m$  and  $l$ , the value for  $i_{w_m}$  in  $T$  is still the correct value for  $i_{w_m}$  in  $T'$ .

Thus for every non- $\perp$  leaf in  $T$ , we can compute the solution in  $T'$ .

**$\perp$  leaves in  $T$ :** In the reduced system, there are three different types of  $\perp$  leaves:

1. *out of bounds writes*; the writes will still be out of bounds in  $T'$ .
2. *reads without any writes referring to the same location*; since the unused indices do not affect the locations written, there can also be no dependent writes in  $T'$ . All such  $\perp$  leaves in  $T$  remain as  $\perp$  leaves in  $T'$ .
3. *reads with no preceding dependent writes*; the fact that there was no preceding dependent write in  $T$  does not necessarily imply that there is no preceding dependent write in  $T'$ . In this category, there is an iteration  $\vec{i}_w$  that writes to the location read in iteration  $\vec{i}_r$  in the fully reduced system, except that the write comes after the read.

If there exists an  $l < k$  such that  $i_{w_l} > i_{r_1}$ , regardless of the value of  $i_{w_k}$  or any of the other previously restored indices, the write will still come after the read and these leaves remain  $\perp$  leaves in  $T'$ .

If no such  $l$  exists, then  $i_{w_1} = i_{r_1}, \dots, i_{w_{k-1}} = i_{r_{k-1}}$  for all the existing indices in  $T$ . In this case, as long as  $i_{w_k} < i_{r_k}$ , there is a preceding dependent write in  $T'$ . Since our read and write bounds are identical, if  $i_{r_k}$  is greater than its lower bound  $LB_k(i_{r_1}, \dots, i_{r_{k-1}})$ , the latest write occurs when  $i_{w_k} = i_{r_k} - 1$ , otherwise there is no  $i_{w_k} < i_{r_k}$  and the solution remains  $\perp$ .

Our non-self-interfering write algorithm makes it easy to find all the leaves where no such  $l$  exists. We merely search  $T$  for the node with the constraint  $i_{w_{k'}} \geq i_{r_{k'}}$ , where  $k'$  is the last used index less than  $k$ . We call this node our *dividing node*. If there is no used index,  $k' < k$ , our dividing node is the last dependence node,  $(PG\vec{u}_r)_x \% D_{x,x} = 0$ , the parent of the first lexicographic constraint node in  $T$ . From the context of this node, we know that  $i_{w_1} = i_{r_1}, \dots, i_{w_{k-1}} = i_{r_{k-1}}$  for all the existing indices in  $T$ . Since every node inherits the context of its ancestors, these equality conditions are

true for every descendent of the true edge of the dividing node. Thus, for any descendent  $\perp$  node of the true edge of our dividing node, no such  $l$  exists. We therefore replace all such descendent  $\perp$  nodes with the subtree whose root is the condition “if  $i_{r_k} > LB_k(i_{r_1}, \dots, i_{r_{k-1}})$ ” and whose false child is a  $\perp$  node. The true child is the solution node with  $i_{w_k} = i_{r_k} - 1$ , every other unused variable set to its respective upper bound and every used variable set to its corresponding component in the solution,  $QD^{-1}PG\vec{u}_r$ . Note that the subtree we have described is the same as the tree generated by our loop-independent algorithm for the case of a read statement followed by a write statement nested in one loop. Loop-independent references correspond to the case when all variables are unused.

### 3.3.4 Example of Restoring Unused Variables

We now illustrate our restoration algorithm with an example. Suppose the example program has a read followed by a write statement, both nested within the same three loops. The second loop index is unused in the write access function and has constant lower and upper bounds,  $LB_2$  and  $UB_2$ , respectively. Assume that after moving the second loop index, the write in the reduced system does not self-interfere. The LWT for the reduced system is shown in Figure 5. We label some of the internal nodes  $N_0$ ,  $N_1$  and  $N_2$  to aid in the discussion.

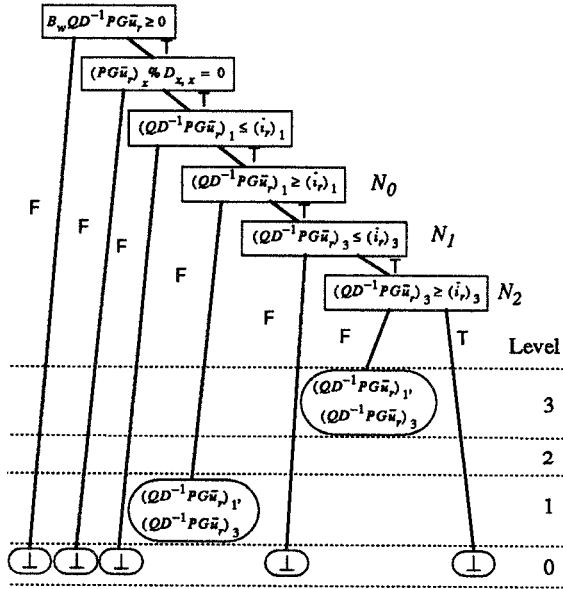


Figure 5: LWT for reduced system

We now wish to restore the second loop index, that is,  $k = 2$ . Consider first the two non- $\perp$  leaves. The

dependence level of the lower left leaf is 1 which is less than  $k = 2$ . We therefore set  $i_{w_2} = UB_2$  for this leaf. The dependence level of the upper right leaf is 3 which is greater than  $k = 2$ . We therefore set  $i_{w_2} = i_{r_2}$  for this leaf.

Now consider the  $\perp$  leaves. Node  $N_0$ , representing the condition that  $i_{w_1} \geq i_{r_1}$ , is the dividing node. For any  $\perp$  node descended from its true edge, we know that  $i_{w_1} = i_{r_1}$ . There are two such  $\perp$  leaves: the false child of  $N_1$  and the true child of  $N_2$ . We replace both  $\perp$  leaves with the subtree whose root is the condition “if  $i_{r_2} > LB_2$ ”, whose false child is a  $\perp$  node, and whose true child has the standard partial solution and  $i_{w_2} = i_{r_2} - 1$ . We show the LWT for the full system in Figure 6.

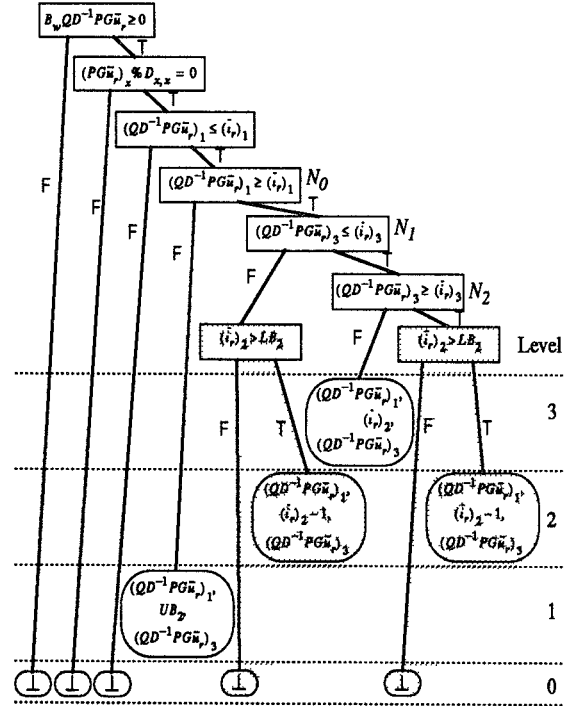


Figure 6: LWT after restoring unused variables

### 3.3.5 Complexity of Algorithm

Our technique offers substantial improvement in time over Feautrier’s algorithm for those problems that are within our algorithm’s domain (as discussed in Section 3.3.1).

The prepass of our algorithm needs to calculate the Smith normal form for the write array access function. Also, in the rare cases where partial degeneracy may occur, the algorithm eliminates the degeneracies using Fourier-Motzkin elimination, which is exponential in the worst case.

The tree built by our algorithm has  $O(n^2)$  nodes where  $n$  is the number of loops in the nest. The tree for the fully reduced system represents the lexicographic ordering constraints with at most  $2n$  internal nodes, and  $2n+1$  leaf nodes. When incorporating an unused index into the system, the only possible change to the tree structure is the replacement of  $\perp$  leaves by subtrees consisting of one internal node, one non- $\perp$  leaf and one  $\perp$  leaf. Since the number of  $\perp$  leaves remains unchanged, the final tree has  $O(n^2)$  nodes. Our algorithm visits each node at most  $n$  times and the computation per node is trivial. In contrast, Feautrier's algorithm can generate exponential trees, where each node involves solving a parametric integer program, which in turn is exponential in complexity.

### 3.4 LWTs for Multiple Reads and Writes

We have described an algorithm to calculate an LWT for a single pair of read and write statements. For multiple read statements we can treat them as separate problems, and construct an LWT for each read statement. This is not possible when dealing with multiple writes.

One approach is to find the LWT for each individual write statement with respect to the same read statement, and find the last of all the last write solutions from the different trees. A description of this algorithm can be found in [15]. The complexity of the resulting tree and therefore the complexity of our algorithm can grow exponentially with the number of write statements. There are, though, two special cases that we believe are common. If two write statements write to exactly the same locations on each iteration, then one write statement always overwrites the writes of the other statement. The desired result is then simply the LWT for the covering write. The other simple case is when the two writes write to non-overlapping locations. The non- $\perp$  leaves in the two LWTs refer to different locations and thus must have disjoint read iteration sets. The solution is simply the union of the two trees.

## 4 Array Privatization

It is important that the information generated by a complex program analyzer be both sufficient and necessary for program optimizations. We now use the example of array privatization to demonstrate that array data-flow information is indeed useful. Recent research by Eigenmann et al.[7], by Singh et al.[22] and by us[17] has shown that array privatization is a critical transformation in parallelizing real programs. Eigenmann et al. discuss the techniques used to hand-parallelize four of the PERFECT Club programs[8]. They find that array privatization is useful in all of the programs. In two of the four programs, 98% of the dynamic execution time of the

program is spent in loops that require array privatization: before they can be parallelized.

In the problem domain of nested loops with affine loop bounds and array indices, Feautrier developed an algorithm that expands the array fully so that no single location is ever written multiple times[9]. Such expansion eliminates all the spurious true dependences (those that are not also data-flow dependences), anti- and output dependences in the loop nest. However, it is too expensive both in execution time and compilation time to fully expand an array indiscriminately.

We have developed a parallelization algorithm that uses array privatization if and when necessary. Our algorithm will only privatize an array when it is legal and when it is necessary to improve parallelism. The cost of privatization is the extra memory required to store separate copies of the array. It is only necessary to store one copy per processor, as opposed to one copy per iteration for array expansion. This minor space overhead is more than offset by the increased parallelism. This is confirmed by Eigenmann et al.'s study which shows significant parallel speedups after hand privatizing real programs[7].

In Section 4.1, we first show why data dependence analysis fails to parallelize loops that reuse arrays. We review the terminology and theory developed using the data dependence analysis results. In Section 4.2, we present an algorithm that combines parallelization with array privatization. We introduce the abstractions of data-flow vectors, upwards exposed uses and downwards exposed definitions within a loop. We show how such information is used in the algorithm. We show how to compute the information from the LWT in Section 4.3. Finally, Section 4.4 shows an example of the code generated by our compiler.

### 4.1 Parallelization with Data Dependence Information

**Definition 4.1** *Let  $s$  and  $s'$  be two array statements in a program. There is a dependence from  $s$  to  $s'$  if there exist two iterations,  $\vec{i}$  and  $\vec{i}'$ , such that*

- *the location accessed by reference  $s$  in iteration  $\vec{i}$  is the same as the location accessed by reference  $s'$  in iteration  $\vec{i}'$ , and*
- *iteration  $\vec{i}$  executes before  $\vec{i}'$ .*

*Iteration  $\vec{i}$  executes before iteration  $\vec{i}'$  if  $\vec{i} \prec \vec{i}'$ , or if  $\vec{i} = \vec{i}'$  and statement  $s$  appears lexically before statement  $s'$  in the program text.*

*The dependence is*

- *a true dependence if  $s$  is a write and  $s'$  is a read;*
- *an anti-dependence if  $s$  is a read and  $s'$  is a write;*

```

for j = 1 to N1 by 2 do
  for i = 1 to N2P do
    ii = i + i
    work[ii - 1] = A[i][j]
    work[ii] = A[i][j + 1]
  for i = 1 to N2P do
    A[i][j] = work[i]
    A[i][j + 1] = work[i + N2P]

```

Figure 7: Code from PERFECT Club Benchmark OCS

- an output dependence if  $s$  and  $s'$  are both write accesses.

**Definition 4.2** The direction vector  $\vec{d}$  from  $s$  to  $s'$  is of length  $m$ , where  $m$  is the common nesting level of the references. For all  $k, 1 \leq k \leq m$ ,

$$d_k = \begin{cases} '+' & i'_k - i_k > 0 \\ 0 & i'_k - i_k = 0 \\ '-' & i'_k - i_k < 0 \end{cases}$$

**Definition 4.3** A dependence vector  $\vec{d} = (d_1, \dots, d_m)$  is said to be carried at level  $k$  if  $(d_1, \dots, d_{k-1}) = (0, \dots, 0)$  and  $d_k$  is '+' [1].

**Theorem 4.1** The  $k^{\text{th}}$  outer loop of a loop nest is parallelizable if the loop nest has no  $k$ -level dependence vectors.

Consider the code extracted from the PERFECT Club benchmark program OCS shown in Figure 7. We consider only dependences involving the *work* array here since the references to the *A* array are parallelizable using existing techniques. Dependence analysis detects:

- true dependences from each write reference *work*[ii-1] and *work*[ii] to each read reference *work*[i] and *work*[i+N2P]: ('+') and (0)
- an anti-dependence from each read reference to each write reference: ('-')
- an output dependence from each write reference to itself: ('+', 0)

By theorem 4.1, the outermost loop is not parallelizable because all the dependences are carried at level 1.

## 4.2 Parallelization with Data-Flow Information

### 4.2.1 Data-Flow Dependence Vectors

Analyzing the data-flow of the example program, it is clear that the only definitions that reach the reads are

those within the same outer iteration. The actual true dependences from the write operations to the read operations are more accurately described by only the direction vector (0). Thus after eliminating the spurious true dependences ('+'), this loop is parallelizable if the artificial anti- and output dependences are eliminated by privatization.

**Definition 4.4** There is a data-flow dependence from a write reference  $w$  to a read reference  $r$  if at least one instance of  $r$  reads a value written by  $w$ .

We can define *data-flow direction vectors* accordingly. Data-flow direction vectors are a subset of true dependence vectors. Those true dependence directions that are not flow directions are artificial dependences that can be eliminated by renaming.

### 4.2.2 Privatizing an Array

We say that an array is privatizable within a loop if there is no need to communicate any of the values written to the array between iterations of the loop.

**Definition 4.5** An array is privatizable in loop  $k$  if the loop has no  $k$ -level data-flow vectors.

Obviously, a loop  $k$  is not parallelizable if any of the arrays is not privatizable in loop  $k$ .

While no communication is necessary between iterations of the parallelized loop, an iteration may be reading values defined before the loop, and the values written may be used after the loop. Therefore, each processor may need to initialize its private copy and to copy back some data from the private copy to the original variable at the end of the execution. We refer to the former process as *initialization* and the latter as *finalization*.

A read of location  $l$  in an iteration  $n$  is *upwards exposed* if iteration  $n$  does not contain a definition (a write) of  $l$  before the read. A write of  $l$  in an iteration  $n$  is *downwards exposed* if it is the last definition of  $l$  in iteration  $n$ . Obviously, if an array is privatizable in a loop, any read that is upwards exposed within an iteration must also be upwards exposed in the enclosing loop. Each iteration must initialize its local copy with values from the global version of the array at all those locations for which there is an upwards exposed use. Finalization is slightly more complicated. Multiple iterations in a loop may write to the same location; only the last iteration that writes to the location should update the global copy.

**Definition 4.6**

Let  $A$  be an array variable, and  $\vec{i}$  be an iteration

$$INIT(A, \vec{i}) = \left\{ \vec{l} \text{ such that } A[\vec{l}] \text{ is an upwards exposed use in iteration } \vec{i} \right\}$$

$$FIN(A, \vec{i}) = \left\{ \vec{l} \text{ such that } A[\vec{l}] \text{ is a definition in iteration } \vec{i} \text{ and it is downwards exposed in the enclosing loop} \right\}$$

### 4.2.3 Algorithm

**Theorem 4.2** *The  $k^{\text{th}}$  outer loop of a loop nest is parallelizable if the loop has no  $k$ -level data-flow dependence vectors, and all arrays with  $k$ -level anti- and output dependences are privatized.*

The algorithm to parallelize a loop with array privatization optimizations is as follows:

1. Compute the data-flow vectors.
2. Find all the parallelizable loops, loops that carry no data-flow dependences. We may choose to parallelize all or just some of the loops.
3. For each loop  $k$  to be parallelized, compute the INIT and FIN sets for each array that has  $k$ -level anti- and output dependences.
4. Modify the code by annotating the loop as a parallelizable loop. For each array privatized, we declare a new local array, insert the code to initialize the INIT set at the beginning of the iteration, and write back the FIN set at the end of each iteration. In practice, multiple iterations can be merged together and scheduled on the same processor.

## 4.3 Using LWTs for Array Privatization

### 4.3.1 Data-Flow Dependence Vectors

Our algorithm calculates LWTs on a dependence level by dependence level basis. All the data-flow dependence pairs described by a leaf are at the same level. This is a useful property for calculating data-flow dependence vectors. Different algorithms for constructing LWTs might not guarantee that each leaf only describes single-level dependences, but it is always possible to split a multi-level leaf into several nodes with only single-level leaves.

We can use an LWT to easily calculate the data-flow dependence vectors. We visit every leaf in the tree that is not labeled  $\perp$ . The context and the solution of the leaf impose a set of linear constraints on the iteration variables which can be expressed in matrix form as  $A\vec{i} \geq 0$  where  $\vec{i}$  is the combined vector  $(\vec{i}_w, \vec{i}_r, \vec{s})$ . This is exactly the same formulation as for the affine memory disambiguation problem. We can solve for the data-flow vectors in the same way that dependence analysis solves for direction vectors using a variation of Burke and Cytron's method[6]. We then return the set union of all the data-flow dependence vectors calculated at each leaf.

The properties of our LWTs can be exploited in optimizing this procedure. Burke and Cytron's method works by enumerating and verifying possible direction vectors. If the dependence level of a leaf is  $k$ , any data-flow dependence vector with a direction other than 0 in any of its first  $k - 1$  dimensions is infeasible. We can immediately prune away any such vector. Given an LWT, we

can calculate data-flow dependence vectors this way as efficiently as calculating direction vectors.

### 4.3.2 INIT and FIN Sets

We can use our LWT directly to calculate the INIT sets. A leaf at the  $k^{\text{th}}$  level indicates the presence of a use that is upwards exposed in the iterations of the  $k^{\text{th}}$  loop. If loop  $k$  is parallelizable, by definition, there are no  $k$ -level leaves in the LWT. All leaves of level less than  $k$  correspond to upwards exposed uses in the iterations of the privatized loops. The INIT set is therefore the access locations of the leaves of these levels. The leaves with level greater than  $k$  do not contribute to the INIT set.

It is then easy to generate code that correctly initializes all the access locations of a leaf in the LWT. We generate a copy of the original set of loops and a reference with the original read access function. We nest the reference within a set of conditional statements so that the reference is made only if the current loop indices satisfy the context of one of the given leaves. Ancourt shows how to optimize this basic algorithm using an extended version of Fourier-Motzkin pair-wise elimination[3]. Ancourt also shows how to eliminate the duplicate access locations in multiple contexts.

The FIN set is not directly available from an LWT. Finding the FIN set, the last write to every location, is in fact a subproblem in calculating the LWT. However, when the LWT is generated using our simplified algorithm, finding FIN sets is trivial. When privatizing the  $k^{\text{th}}$  loop:

- If  $i_k$  is used, definitions of different iterations of  $i_k$  are independent. Therefore  $FIN(A, (i_1 \dots i_k))$  is simply the set of all the definitions in the iteration.
- If  $i_k$  is unused, definitions of different iterations of  $i_k$  are the same. Thus  $FIN(A, (i_1 \dots i_k))$  is the set of definitions in the given iteration if  $i_k = UB_k$ , and empty otherwise.

## 4.4 Implementation and An Example

We have implemented our algorithm in the SUIF compiler system. Our parallelizer marks all the parallelizable loops and privatizes arrays when needed.

We show the output of our compiler on the code in Figure 7 as an example. Besides parallelization and privatization, the SUIF optimizer has to first perform loop normalization, constant propagation and induction variable identification on the code. The code to be run by every processor is shown below. (To simplify the presentation, we do not parallelize the inner loops although they can be parallelized using standard techniques.)

```
LB = 0
UB = floor((N1-1)/2)
```

```

Blk = ceil((UB-LB)/NProc)
if (pid!=NumProc-1) then
  /* not the last processor */
  for j = LB+pid*Blk to LB+(pid+1)*Blk -1 do
    /* _work_priv is a local array */
    for i = 1 to N2P do
      _work_priv[2*i-1] = A[i][2*j+1]
      _work_priv[2*i] = A[i][2*j+2]
    end for
    for i = 1 to N2P do
      A[i][2*j+1] = _work_priv[i]
      A[i][2*j+2] = _work_priv[i+N2P]
    end for
  end for
else /* last processor */
  for j = LB+pid*Blk to UB do
    for i = 1 to N2P do
      work[2*i-1] = A[i][2*j+1]
      work[2*i] = A[i][2*j+2]
    end for
    for i = 1 to N2P do
      A[i][2*j+1] = work[i]
      A[i][2*j+2] = work[i+N2P]
    end for
  end for
end for

```

Note that it is not easy to discover that initialization is unnecessary for this example. Each read gets part of its data from each write. Therefore, we cannot simply look at pairs of references. The effects of the two writes must be combined by merging the two individual LWTs. Since the two writes write to non-overlapping locations, our system efficiently determines that initialization is unnecessary. As for finalization, since all the iterations of the loop write to the same locations, the final values of the array are determined only by the last iteration. Since no initialization is needed, the last processor can directly operate on the original copy of *work* and no synchronization is necessary.

## 5 Conclusions

Researchers in compiler technology have long realized that flow-insensitive alias analysis, is not sufficient for advanced optimizations. It is necessary to analyze the flow of data through the program. Similarly, we believe that parallelizing compilers have now reached the stage where data dependence analysis, the array analog to scalar alias analysis, is no longer sufficient for more advanced optimizations.

Previous techniques of array data-flow analysis are either too inefficient or too imprecise. We have developed an efficient algorithm that can find perfect information in the domain of loop nests with affine loop bounds and

array indices in most of the programs found in practice. The efficiency of this algorithm is important in making array data-flow analysis feasible to use in program optimizations.

We have applied this analysis to the problem of array privatization, itself an optimization that has been established to be important. This optimization demonstrates the need for information on coverage and the identity of dependence, both missing in standard data dependence analysis. This paper provides the basic technology for array privatization. Extension of the analysis across procedures is necessary to make it truly applicable to real programs.

More important than a particular optimization is the general framework of our approach. In developing the algorithm for array privatization, we have defined several useful higher level abstractions: data-flow dependence vectors, upwards-exposed uses and downwards-exposed definitions of individual array elements. These abstractions can be derived easily from the Last Write Tree, the low-level representation for the array data-flow information. These concepts will make possible many more advanced optimizations such as minimizing communication on a distributed memory machine.

## References

- [1] R. Allen, D. Callahan, and K. Kennedy. Automatic decomposition of scientific programs for parallel execution. Technical Report Rice COMP TR86-42, Department of Computer Science, Rice University, Nov 1986.
- [2] C. Ancourt and F. Irigoin. Scanning polyhedra with do loops. In *Proceedings of the ACM SIGPLAN 1991 Principles and Practice of Parallel Programming*, pages 39–50, 1991.
- [3] M. Ancourt. *Generation automatique de codes de transfert pour multiprocesseurs a memoires locales*. PhD thesis, University of Paris VI, March 1991.
- [4] U. Banerjee. *Dependence Analysis for Supercomputing*. Kluwer Academic, 1988.
- [5] T. Brandes. The importance of direct dependences for automatic parallelism. In *Proceedings of 1988 International Conference on Supercomputing*, pages 407–424, 1988.
- [6] M. Burke and R. Cytron. Interprocedural dependence analysis and parallelization. In *Proceedings of the SIGPLAN 1986 Symposium on Compiler Construction*, pages 162–175, 1986.
- [7] R. Eigenmann, J. Hoeflinger, Z. Li, and D. Padua. Experience in the automatic parallelization of four

- perfect-benchmark programs. In *Languages and Compilers for Parallel Computing*, pages 65–83, Aug 1991.
- [8] M. Berry et al. The PERFECT Club benchmarks: effective performance evaluation of supercomputers. Technical Report UIUCSRD Rep. No. 827, University of Illinois Urbana-Champaign, 1989.
  - [9] P. Feautrier. Array expansion. In *International Conference on Supercomputing*, pages 429–442, 1988.
  - [10] P. Feautrier. Parametric integer programming. Technical Report 209, Laboratoire Methodologie and Architecture Des Systemes Informatiques, Jan 1988.
  - [11] P. Feautrier. Dataflow analysis of array and scalar references. *International Journal of Parallel Programming*, 20(1):23–52, Feb 1991.
  - [12] E. D. Granston and A. V. Veidenbaum. Detecting redundant accesses to array data. In *Supercomputing '91*, pages 854–865, 1991.
  - [13] T. Gross and P. Steenkiste. Structured dataflow analysis for arrays and its use in an optimizing compiler. *Software - Practice and Experience*, 20(2):133–155, 1990.
  - [14] Z. Li. Array privatization for parallel execution of loops. In *International Conference on Supercomputing*, 1992.
  - [15] D. E. Maydan. *Accurate Analysis of Array References*. PhD thesis, Stanford University, October 1992.
  - [16] D. E. Maydan, J. L. Hennessy, and M. S. Lam. Efficient and exact data dependence analysis. In *Proceedings of the SIGPLAN 1991 Programming Language Design and Implementation*, pages 1–14, 1991.
  - [17] D. E. Maydan, J. L. Hennessy, and M. S. Lam. Effectiveness of data dependence analysis. In *Proceedings of the NSF-NCRD Workshop on Advanced Compilation Techniques for Novel Architectures*, 1992.
  - [18] W. Pugh. The omega test: a fast and practical integer programming algorithm for dependence analysis. In *Supercomputing '91*, pages 4–13, 1991.
  - [19] W. Pugh and D. Wonnacott. Eliminating false data dependences using the omega test. In *Proceedings of the SIGPLAN 1992 Programming Language Design and Implementation*, pages 140–151, 1992.
  - [20] H. Ribas. Obtaining dependence vectors for nested-loop computations. In *Proceedings of 1990 International Conference on Parallel Processing*, pages II–212 to II–219, 1990.
  - [21] C. Rosend. *Incremental Dependence Analysis*. PhD thesis, Rice University, March 1990.
  - [22] J. P. Singh and J. L. Hennessy. An empirical investigation of the effectiveness and limitations of automatic parallelization. In *Proceedings of the International Symposium on Shared Memory Multiprocessing: Tokyo, Japan*, pages 25–36, 1991.